Lezione 1- Componenti Funzionali in React React

Introduzione

Abbiamo visto nella scorsa lezione come Installare Node.JS, la struttura di un progetto e come far partire un applicazione React, ora vedremo come scrivere codice JS.

Componenti Funzionali

React si basa sulla creazione e combinazione di **componenti riutilizzabili e reattivi**.
Inizialmente, React utilizzava componenti basati su classi, ma con l'introduzione degli **Hooks** in React 16.8, i **componenti funzionali** sono diventati più popolari per la loro concisione e la gestione semplificata dello stato e del ciclo di vita.

Come funziona React

React utilizza un **Virtual DOM** per aggiornare l'interfaccia utente in modo efficiente.

1. Quando lo stato o le props di un componente cambiano, React crea una **rappresentazione virtuale del DOM**.
2. Questa versione viene confrontata con la precedente per calcolare le differenze (**"diffing"**).
3. Solo le parti del DOM che devono essere modificate vengono aggiornate, evitando di ricaricare l'intero DOM e rendendo gli aggiornamenti molto efficienti.

Creare un Componente React

Per creare un nuovo componente in React, segui questi passaggi:

1. **Crea un nuovo file .js** all'interno della cartella `src`, ad esempio `componente1.js`.

React JSX

```
import React from 'react'
```

```
const Componente1 = ()=>{

    return (
    <div>
        <h1> Il mio primo componente </h1>
    <div>
    )
}
export default Componente1

```

1. **Installa l'estensione ES7 React** nel tuo Visual Studio Code.
2. Utilizza il comando `rafce` (React Arrow Function Export Component) del plugin per generare lo **scheletro del componente**.
3. Scrivi il contenuto desiderato all'interno del `div` del componente.
4. **Richiama il componente** all'interno del componente principale `App.js` importandolo e utilizzandolo come un tag HTML, ad esempio `<Componente1/>` o `<Componente1></Componente1>`.

React JSX

```
import React from 'react';
import Componente1 from './Componente1.js'
function App(){
    return (
        <div>
            <Componente1 />
        </div>
    )
}
export default App
```

I due modi per definiere un componente

In React esistono 2 modi per definire un componente:

1. Funzione Tradizionale:

React JSX

```
function MyComponent (){
//corpo del componente
}
```

```
4
```

2. Arrow Function

React JSX

```
const MyComponent = ()=> {
    //corpo del componente
}

```

Entrambi i modi sono corretti:

- la **function tradizionale** è il vecchio modo di scrivere un componente React;
- la **arrow function** è oggi lo standard, più conciso e utilizzato da snippet come `rafce`.

Inoltre i componenti devono **sempre iniziare con la lettera maiuscola** (`Componente1`, non `componente1`), altrimenti React li interpreterà come **tag HTML** e non come componenti personalizzati.

La sintassi JSX

Prendiamo ad esempio questo codice:

React JSX

```
import React from 'react';
import './App.css';
import Componente1 from './Componente1';

function App(){
    return (
        <div className = "App">
            <h1> Componente Principale </h1>
            <Componente1 />
        </div>

    )
}

```

```
export default App;

```

Come possiamo notare all'interno dello statement `return` il codice sembra HTML, ma in realtà è codice JSX:
JSX è il linguaggio che definisce il markup HTML da restituire per il rendering visuale all'interno della pagina.
In sostanza JSX inietta codice JavaScript in HTML, quindi quello che ritorniamo nel corpo dello statement `return` sono elementi JSX (cioè i tag HTML) che verranno renderizzati sulla nostra web browser page.
Ci sono alcune regole da considerare, quando si utilizza la sintassi JSX:

1. perché JSX funzioni, è necessario wrappare tutti gli elementi in un singolo tag.
   - In altre parole all'interno del return deve sempre esserci un outer element (o outer tag) che fa da wrapper per gli altri elementi annidati al suo interno

React JSX

```
import React from 'react';
import Componente2 from './Componente2'

const MyComponent = ()=>{
    return(
        <> //è buona pratica utilizzare, come outer elements, il tag div o i React.fragment
            <h2>Questo è l'header</h2>
            <Componente2 />
            <input />
            <p></p>
            ...
        </>
    )
}
```

2. Per richiamare una classe CSS dobbiamo usare la parola chiave className invece di class (poiché in JS, class è una parola riservata).
3. Chiudere tutti i tag anche se singoli (vedi `<Componente2 />` o `<br />`).

Componenti innestati

In **React** è possibile **innestare più componenti** fra loro.
Nell'esempio precedente, l'elemento `<Componente2 />` era un componente

importato e innestato all'interno di `MyComponent`.
Per comprendere meglio il concetto, osserviamo l'esempio seguente:

React JSX

```react_jsx
import React from 'react';

const Componente1 = ()=>{
    return (
        <div>
            <h1> Il mio primo componente</h1>
            <Anagrafica />
            <Messaggio />
        </div>
    )
}

const Anagrafica = ()=> {
    return (
        <h2> Il mio nome è Pinko Pallo</h2>
    )
}


const Messaggio = ()=> {
    return (

        <p>Benvenuti!</p>
    )
}

export default Componente1;

```

Spiegazione:

Nel codice sopra, `Componente1` **contiene** (`innesta`) altri due componenti:

- `<Anagrafica />` e `<Messaggio />`.

Ogni componente è indipendente, ma quando `Componente1` viene **renderizzato** all'interno del `return` di `App.jsx`, React esegue la logica e il rendering anche di `Anagrafica` e `Messaggio`, mostrandoli tutti insieme sullo schermo.

Componenti in file separati

Nella pratica, ogni componente viene di solito salvato nel **proprio file**:

CSS

```
src/
 ├── App.jsx
 ├── Componente1.jsx
 ├── Anagrafica.jsx
 └── Messaggio.jsx
```

In questo caso, `Componente1.jsx` importerà gli altri due componenti:

React JSX

```jsx
import React from 'react'
import Anagrafica from './Anagrafica'
import Messaggio from './Messaggio'

const Componente1 = () => {
  return (
    <div>
      <h1>Il mio primo componente</h1>
      <Anagrafica />
      <Messaggio />
    </div>
  )
}

export default Componente1;
```

Tuttavia assumiamo che questi tre componenti siano nello stesso file e che si voglia esportarli tutti i tre insieme:

- In **React (e in JavaScript in generale)**, in un file può esserci **solo un** `export default`.
- Tuttavia, è possibile esportare anche **altri elementi** (funzioni, componenti, costanti, ecc.) tramite `export` nominato.

React JSX

```jsx
// stesso file
```

```
export default Componente1
export {Anagrafica, Messaggio}
```

- Di conseguenza l'import in altri file come `App.jsx` sarà:

React JSX

```
import Componente1, {Anagrafica, Messaggio } from
'./percorso/del/file';
```

- `Componente1` viene importato come **default export**
- `Anagrafica` e `Messaggio` vengono importati come **named exports**

Tip

Riepilogo

- ✅ Un file può avere **un solo** `export default`
- ✅ Può avere **più** `export` nominati (`export { … }`)
- ✅ Quando si importa:
- `import Nome from 'file'` → importa il *default*
- `import { Nome } from 'file'` → importa un *named export*

Variabili in un componente

All'interno di un **componente funzionale**, puoi dichiarare variabili in JavaScript (come `const` , `let` o `var`) e poi **usarle dentro il JSX** — cioè nel codice che React renderizza sullo schermo.

Come si fa?

Nel JSX, per "entrare" nel contesto JavaScript si usano **le parentesi graffe** `{ }` . Dentro le graffe puoi mettere qualsiasi **espressione JavaScript valida**: variabili, funzioni, operazioni aritmetiche, ecc.

Esempio :

React JSX

```
import React from 'react'

```

```
const Messaggio = () => {
  const nome = "Marco"
  const anno = 2025
  const saluto = `Ciao ${nome}, benvenuto in React!`

  return (
    <div>
      <h1>{saluto}</h1>
      <p>Siamo nel {anno}</p>
    </div>
  )
}

export default Messaggio

```

Spiegazione:

- `{saluto}` → mostra il valore della variabile `saluto`
- `{anno}` → mostra il valore della variabile `anno`
- le graffe `{ }` dicono a React: “questo non è testo, ma codice JavaScript da valutare”

Style inline in React

In React puoi applicare uno **stile inline** come in HTML, ma con una **differenza importante**:
lo stile va scritto come **oggetto JavaScript**, non come stringa.

⚠️ Differenza da HTML

In HTML scriveresti:

HTML

```
<h1 style="color: red; font-size: 20px;">Ciao</h1>
```

In React invece devi scrivere:

React JSX

```
<h1 style={{ color: 'red', fontSize: '20px' }}>Ciao</h1>
```

Perché le doppie graffe?

- **La prima coppia di graffe `{ }` serve per entrare nel contesto JavaScript dentro il JSX.**
- **La seconda coppia `{ }` definisce l'oggetto CSS vero e proprio.**

Ecco perché scrivi `style={{ color: 'red' }}` → una graffa "apre" il JavaScript, e dentro c'è un oggetto con le regole CSS.

Esempio completo

React JSX

```react-jsx
import React from 'react'

const Titolo = () => {
  const stileTitolo = {
    color: 'blue',
    backgroundColor: 'lightgray',
    padding: '10px',
    borderRadius: '8px'
  }

  const nome = "React"

  return (
    <div>
      <h1 style={stileTitolo}>Ciao, sono {nome}!</h1>
      <p style={{ fontStyle: 'italic', color: 'gray' }}>
        Questo è uno stile inline.
      </p>
    </div>
  )
}

export default Titolo

```

Note didattiche:

- `stileTitolo` è un **oggetto JavaScript** contenente regole CSS.
- Ogni proprietà CSS scritta con `-` (es. `font-size`) diventa **camelCase** in React (`fontSize`).

- I valori vanno sempre tra **apici singoli o doppi** (`'blue'` , `'10px'`).